

WEXSPACE AI

Agents for Humans · Professional Agents · Implementation architecture

A governed receiving-evidence workflow. Synthetic inputs: 120 expected units, 118 observed units. Deterministic reconciliation produces a discrepancy of -2 units and a human review package.

1 · Request compiler

Objective and required inputs form the execution contract.

2 · Strands interface

Native SDK tools with ordered dispatch, saved named results, and a configured Bedrock route.

Judge UI / API

Loopback Python server calls the same deterministic core. Synthetic demo mode performs no model inference.

3 · Governed deterministic runtime

inspect_request → reconcile_evidence → prepare_review_package

4 · SQLite state

work_id, node outputs, durable cursor, and unique effect keys.

Evidence ledger

Canonical JSONL records with SHA-256 links; exported decision and evidence receipts.

5 · Human review boundary

State becomes HUMAN_REVIEW. Agent tools and the demo API expose no approval endpoint.

Interruption and continuation

After inspection saves cursor 1, the qualification process kills the Python server with SIGKILL. A different Python process opens the same state directory. The browser reconnects to the same work ID, resumes remaining safe tools, and reads the review receipt. Each recorded tool effect occurs once.

Execution surfaces

Surface	Current scope
CLI + Python core	Deterministic workflow, durable state, review export.

Strands SDK	Real direct SDK tool dispatch qualified separately; model inference is a separate gate.
Web UI	Local synthetic product demo; no public multi-user hosting claimed.
AgentCore bridge	Source integration only; deployed runtime and cloud invocation remain unverified.

Trust boundaries and practical limits

- Inputs in the public demonstration are synthetic. No owner credentials or employer data are displayed.
- Persistent state belongs to the runtime. Model text does not override the saved cursor or completed node result.
- The Web server binds to loopback, validates its Host header, and rejects cross-origin browser mutations.
- Production deployment needs a verified cloud identity, authorized resources, authenticated user access, and an appropriate durable state service.
- The process test kills after cursor publication. It does not prove atomicity inside an external distributed side effect or concurrent multi-writer execution.
- Hash-linked evidence establishes integrity within the recorded chain; it is not independent third-party attestation.
- The recorded display segment is continuous X11 capture. It is a qualification segment, not the final competition video.

Public reproducibility

The Public source replay workflow installs a fresh Python environment, checks dependencies, runs deterministic, SDK-tool and Web/API tests, executes the README workflow, and verifies a real process interruption.

The Browser and real display qualification workflow starts the actual application, drives Chromium, records the display with FFmpeg, and stores the raw segment, manifest, receipt, and this architecture PDF in a separate evidence branch.

Source commit

46de154b4c1ca684a77b78f1218435757670395a

Browser execution run

34857543370

[Public source repository](#)

Original WEXSPACE implementation · MIT license · Ongoing WEXSPACE program and competition-specific changes are disclosed in the project description.